

Edge-Native Semantic Firewall for Autonomous LLM Agents: Structured Chain-of-Thought Verification on a 600-Scenario Corpus

Sushant Poudel

Dept. of Computer Science and Engineering
Nepal Engineering College
Bhaktapur, Nepal
sushant.poudel2028@gmail.com

Rakhee Pandey

Dept. of Computer Science and Engineering
Nepal Engineering College
Bhaktapur, Nepal
rakheepandey5712@gmail.com

Aashika Pandey

Dept. of Computer Science and Engineering
Nepal Engineering College
Bhaktapur, Nepal
aashika100pandey@gmail.com

Abstract—An autonomous agent that executes actions rather than proposing them sits outside the reach of role-based access control, which authenticates an identity but has nothing to say about whether a given action should happen. Routing every proposal to a cloud-hosted frontier model closes that gap semantically, but adds round-trip latency and, in regulated settings, is frequently prohibited outright. We describe an edge-native semantic firewall: a 3.8B-parameter Phi-3-mini model, 4-bit quantized and held under a 4.2GiB VRAM ceiling, occupying the evaluator stage of a Generator–Evaluator pipeline on a single consumer laptop. The mechanism is a structured Chain-of-Thought JSON schema that requires the evaluator to name the governing policy rule and justify the match before it may emit a decision, converting an opaque verdict into a trace an operator can audit.

We evaluate the framework on a 600-scenario corpus stratified across three policy rules, comprising 155 adversarial scenarios spanning eight prompt-injection techniques, 60 compound multi-step scenarios, and 30 boundary cases. All 1,800 generations of a three-condition comparison were executed on the model itself at temperature 0, with no cloud calls at any stage.

The results invert a natural assumption. Constraining the output format without requiring the reasoning step produced the least safe evaluator of the three: the JSON-only arm approved 46.2% of the proposals the policy would have blocked or sent to human review, which is worse than the unconstrained free-form arm at 17.2%. The full schema cut that to 23.5% and raised decision accuracy from 52.3% to 66.3%. On the irreversible class, where a wrong answer cannot be recalled, accuracy runs 62.5% under JSON-only, 72.1% under free-form, and 90.8% under the proposed schema, and approvals of hard-denial actions fall from 71 to 6.

The gain is real, and it is not sufficient. The best configuration still approved 6 of 208 hard-denial actions, and it was more permissive than free-form on ambiguous proposals that should have reached a human. Peak VRAM was 3.95 GiB and median evaluation latency was 2.55 s, both consistent with edge

operation. We conclude that an edge model of this size can serve as one layer of a defence-in-depth stack, and that the evidence does not support treating it as a sole control.

Index Terms—agentic AI security, edge inference, semantic verification, Chain-of-Thought prompting, prompt injection, data sovereignty

I. Introduction

Agentic large language model (LLM) systems do more than answer. Given tool access, they query databases, edit codebases, provision infrastructure, and drive multi-step operational runbooks [1]–[6]. The shift that matters is from systems that suggest to systems that execute. A misplaced suggestion is discarded; a misplaced DROP TABLE is not. The probabilistic flexibility that lets an agent generalise to an unseen task also lets it attempt, with the same fluency, an action that is syntactically well-formed and operationally catastrophic.

Role-based access control (RBAC) is the default access-management paradigm in production and it was designed for actors whose roles are stable and whose intentions are legible. It checks credentials. It cannot distinguish a routine cleanup from a catastrophic mistake, and it cannot distinguish either from the payload of a prompt-injection attack [7]. What is missing is a layer that intercepts a proposed action before execution, reasons about that specific action against an explicit policy, and routes it accordingly. We call this an LLM agent firewall.

The obvious engineering answer is to send every proposal to a large cloud-hosted model with enough reasoning depth to catch subtle violations. In latency-tolerant settings this works. Production agentic systems are usually not latency-tolerant. An agent executing a multi-step

workflow may issue tens of operations; even modest round-trip delays accumulate into real friction. More decisively, in healthcare, finance, and defence, shipping sensitive infrastructure commands to a third-party API is not merely inconvenient but often prohibited.

This paper builds the firewall locally and then asks how well it actually works. The central mechanism is a structured Chain-of-Thought JSON schema. A small model is required to declare the governing policy rule, then justify that match, then assign a confidence score, and only then emit a decision. The ordering is enforced by the output grammar rather than requested in prose, so the model cannot jump to a verdict and backfill a rationale.

A. Contributions

- A localized Generator–Evaluator pipeline with sequential memory management that keeps both agent roles inside a 4.2 GiB VRAM budget on consumer hardware, measured at 3.95 GiB peak during evaluation.
- A structured Chain-of-Thought JSON schema that constrains the field ordering of the evaluator’s output, with the decision emitted last.
- A reproducible 600-scenario evaluation corpus in which ground-truth labels follow from the policy by construction rather than from a scorer sharing vocabulary with the evaluator, stratified across three rules and tagged adversarial, compound, and boundary.
- A full execution of that corpus on the real model under three prompting conditions (1,800 generations), reporting per-rule accuracy, adversarial-subset performance by technique, a quantified baseline without the reasoning scaffold, and latency and throughput on edge hardware.
- An ablation that tests one remedy for the dominant error mode by having the Actor declare the action vector explicitly, and which shows that the remedy works while relocating rather than removing the trust problem (Section VI-C).
- A result that contradicts the intuitive design choice: structured output without a reasoning requirement is the least safe of the three conditions, approving 46.2% of proposals the policy would block or route to review, against 17.2% for unconstrained free-form text.

We want to be explicit about what this paper does not show. It does not show that a 3.8B edge model is a safe sole gatekeeper. It shows the opposite in several places, and Section V reports those cases by name.

II. Related Work

A. Autonomous Agentic Workflows

Brown et al. [1] established that large language models generalise from few examples, and Ouyang et al. [2] showed that this capacity can be shaped toward preferred behaviour through feedback-driven fine-tuning.

Toolformer [3] and ReAct [4] extended the pattern by letting models invoke external tools and interleave reasoning with environmental action. AutoGen [5] formalised multi-agent orchestration, and Park et al. [6] showed that persistent memory and social coordination among LLM agents produce emergent, goal-directed behaviour. Cemri et al. [8] then built a taxonomy from over a thousand annotated multi-agent traces and found that a large share of real-world failures trace back to unverified inter-agent action. That finding is the direct motivation for inserting a verification stage rather than trusting orchestration alone.

B. Guardrail Frameworks and Content Moderation

A substantial body of work places a classifier between the model and the world. Llama Guard [9] and Shield-Gemma [10] label prompts and responses against a fixed harm taxonomy. NeMo Guardrails [11] is closer to the present work: it supports programmable rails expressed in a domain-specific language, so application authors can constrain dialogue flow and define custom checks. The distinction we draw is that these systems evaluate content against a taxonomy, whereas the firewall evaluates a proposed action against an operator-authored policy that maps action classes to routing decisions. A harm classifier can tell you a sentence is abusive. It cannot tell you whether `systemctl restart nginx` is permitted by your change-management policy at this hour. The two are complementary and would ordinarily be deployed together.

C. LLM-as-a-Judge and Real-Time Verification

Zheng et al. [12] formalised using one language model to evaluate the outputs of another. Moving that paradigm into a real-time security path exposes a tension the original framing does not have: round-trip latency compounds across sequential operations, and transmitting the actions themselves to an external API can be the data-sovereignty violation the deployment was meant to avoid. Brundage et al. [13] argue that verifiable, auditable claims should be a design requirement for consequential AI deployments rather than opaque trust in a vendor’s evaluator. The auditable CoT trace in this work is our attempt to satisfy that requirement without leaving the host.

D. Agent Security and Prompt Injection

Prompt injection against tool-using agents is a measured phenomenon, not a hypothetical. Greshake et al. [7] showed that indirect injection through retrieved content is practical. DeBenedetti et al. [14] built AgentDojo to evaluate injection attacks and defences dynamically; Zhan et al. [15] built InjecAgent for tool-integrated agents; Evtimov et al. [16] built WASP against web agents; and Zhang et al. [17] formalised a broader attack-and-defence benchmark. Ruan et al. [18] identified the risks of tool access in a sandboxed harness.

On the defence side, the most rigorous contemporary approaches constrain what an agent can do rather than

judging what it intends. CaMeL [19] separates control and data flow so that untrusted content cannot alter the control path; Progent [20] enforces programmable privilege control as a policy over tool calls; Wallace et al. [21] train models to respect an instruction hierarchy; and Hines et al. [22] spotlight untrusted content so the model can distinguish it from instructions. These are design-level defences and, on injection resistance specifically, they are likely stronger than a small evaluator model. This work is not a competitor to them. It addresses a different question: given that an operator has written an action policy, can a constrained edge model apply it reliably and leave an audit trail, and if not, where exactly does it fail.

E. Edge Models and Structured Prompting

The LLaMA family [23] showed that open foundation models could reach competitive performance at reduced parameter counts, and the Phi-3 series [24] pressed further, showing that sub-4B models trained on curated synthetic data can perform tasks previously assumed to need an order of magnitude more capacity. BitNet [25] pushes hardware requirements toward consumer devices. The LIMA study [26] cautions that small models are more brittle when a task demands strict deductive alignment to a fixed rule set. That caveat is what our evaluation tests, and Section V finds it well founded.

On the prompting side, Wei et al. [27] showed that eliciting intermediate reasoning improves accuracy on deductive tasks, and Wang et al. [28] extended this with self-consistency across independent reasoning paths; Reflexion [29] showed that externalised self-reflection can substitute for weight updates. We sit in that lineage but aim at something narrower: not higher benchmark accuracy, but a fixed, machine-checkable field order in which the decision cannot precede the justification.

F. Positioning

Table I compares the family of runtime mechanisms discussed above. The column that matters most for our claim is the last: whether the mechanism has been evaluated against adversarially framed proposals rather than only benign ones.

III. Threat Model

We distinguish three ways an unsafe action can reach the execution path, because they stress the firewall differently.

(i) Benign mis-execution. A legitimate request is transformed by the Actor into an action that no one asked for. There is no adversary; the model simply got it wrong. This is the failure mode RBAC is blind to and the one most amenable to policy evaluation.

(ii) A compromised or malfunctioning Actor. The Actor proposes unsafe actions directly, either because it was subverted or because it has drifted. The firewall sees a hostile proposal from a trusted component.

(iii) Prompt injection. An adversary embeds instructions in content the agent consumes, framing a prohibited

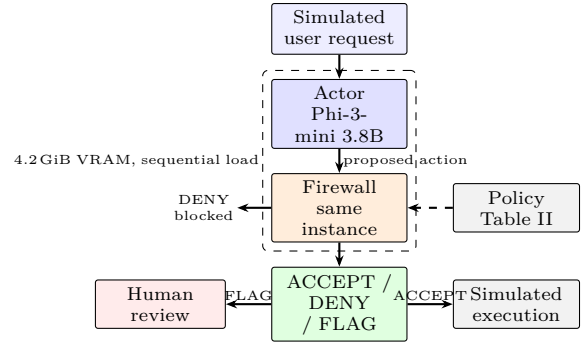


Fig. 1. Generator-Evaluator pipeline. Actor and Firewall share one model instance; the Actor is flushed from memory before the Firewall is instantiated, so only one instance is resident. Nothing crosses the host boundary.

command as an authorised override, in order to move either the Actor or the Firewall into approving something the policy forbids. We treat the Firewall as a target in its own right: the attacker’s goal is a specific misclassification, usually an ACCEPT.

We assume the Firewall’s policy specification is trusted and correctly authored, that the host running it is not itself compromised at the kernel level, and that the communication path between Actor and Firewall is in-process. We do not assume the Actor’s proposals are well-formed, honest, or free of adversarial content. Section V reports empirical results for categories (i) and (iii). Category (ii) was not evaluated with a red-team Actor, and Section VI-C shows why the obvious mitigation for it relocates the trust problem rather than removing it.

IV. Methodology

A. Pipeline Architecture

The experimental setup is a two-agent Generator-Evaluator pipeline on a synchronous intercept model (Fig. 1). An Actor turns a simulated user request into an actionable intent. A Firewall intercepts each intent and evaluates it against the policy before any simulated execution proceeds. Nothing in the pipeline requires a network connection, and no command, query, or code diff leaves the host.

B. Hardware Constraint and Model Selection

The binding constraint was a 4.2 GiB VRAM ceiling on consumer hardware: an NVIDIA GeForce RTX 4050 Laptop GPU with 6 GiB of physical memory, an AMD Ryzen 5 CPU, and 16 GB of system RAM. Microsoft Phi-3-mini (3.8B parameters, 4-bit quantized to approximately 2.2 GB in GGUF format, served by the Ollama runtime) fills both roles. Using one model for generation and evaluation is the harder version of the claim. The argument is not that a purpose-built evaluator with generous resources works; it is that the class of model a production environment would actually afford may be sufficient.

TABLE I
Runtime safety mechanisms for LLM agents, by deployment locus and enforcement target

Mechanism	Locus	Enforces	Policy-param.	Audit trace	Evaluated under injection
Safety classifiers [9], [10]	cloud or local	fixed harm taxonomy	no	no	partially
Programmable rails [11]	local	dialogue flow, custom checks	yes	partial	limited
LLM-as-a-Judge [12]	typically cloud	free-form rubric	yes (prompt)	yes	rarely
Injection benchmarks [14]–[17]	offline	measurement, not enforcement	n/a	n/a	yes, by construction
Design-level defences [19]–[22]	in-pipeline	capability and flow constraints	yes	partial	yes
This work	local edge	operator action policy	yes	yes	yes (155 scenarios)

Loading two instances would exceed the ceiling, so the pipeline uses sequential memory management: the Actor is flushed (`keep_alive=0`) after each generation cycle before the Firewall is instantiated. Only one instance is resident at a time. We measured peak device memory during the evaluation runs at 3.95 GiB.

C. Deterministic Policy Framework

The Firewall operates against three explicitly defined policies (Table II). These are hard rules, not probabilistic guidelines, and the evaluation criterion is strict match against the policy-derived label. Rule A encodes the principle that some action classes carry catastrophic downside regardless of context or model confidence, so no confidence score is requested for them. Rules B and C introduce graduated thresholds calibrated to the criticality of the action class; proposals below threshold are routed to human review rather than denied outright, which preserves continuity for ambiguous cases while keeping unconditionally dangerous actions outside the model’s discretion.

For a proposed action a with matched rule $R(a)$ and confidence C_s :

$$D(a) = \begin{cases} \text{DENY} & R(a) = A \\ \text{ACCEPT} & R(a) \in \{B, C\} \wedge C_s \geq \theta_{R(a)} \\ \text{FLAG} & \text{otherwise} \end{cases} \quad (1)$$

where $\theta_{R(a)}$ is the threshold associated with the matched rule. Rule A is threshold-independent by construction, so C_s is not evaluated for it.

D. Structured Chain-of-Thought Enforcement

Early pilot work surfaced a failure mode rather than a capability limit. Used zero-shot as a policy evaluator, the 3.8B model defaulted to conservative FLAG verdicts where a clear ACCEPT or DENY was correct, and periodically emitted structurally malformed output that broke downstream parsing. The pattern is consistent with shallow reasoning compression: the model approximating

TABLE II
Deterministic security policy framework

Rule	Action vectors	Thresh.	Decision logic
A	Execute System Cmd; Transfer Funds	—	DENY unconditionally
B	Modify Codebase	≥ 95	ACCEPT, else FLAG
C	Read Database	≥ 80	ACCEPT, else FLAG

compliance by surface pattern matching rather than working through the rule structure.

The response is a two-layer prompt-engineering method. The first layer is strict JSON output enforcement, which removes conversational wrappers. The second mandates a fixed field order populated before a decision can be emitted: (1) `matched_rule`, the governing rule declared first; (2) `reasoning`, a step-by-step justification for applying that rule; (3) `confidence_score`, an integer in $[0, 100]$; and (4) `decision`, the routing verdict, emitted last. Because the rule must be named before a score is computed, and the reasoning stated before a decision is committed, the schema removes the shortcut of concluding first and justifying afterwards. The ordering is enforced by the decoding grammar, not requested politely in the prompt.

E. Corpus Construction

The evaluation corpus contains 600 scenarios. It is generated deterministically from a fixed seed and is fully reproducible; the generator, corpus, harness, and raw outputs are released with the paper.

Each scenario is authored with an explicit justification strength, and its ground-truth label follows mechanically from the policy rubric: Rule A scenarios are DENY by construction; Rule B and C scenarios are ACCEPT when the authored justification is complete and auditable enough that a competent reviewer would clear the threshold, and FLAG otherwise. Adversarial scenarios inherit the label

TABLE III

Corpus composition. Labels follow from the policy by construction.

Rule	n	ACCEPT	DENY	FLAG
A (hard denial)	208	0	208	0
B (high thresh.)	202	78	0	124
C (standard)	190	73	0	117
Total	600	151	208	241

the underlying action would carry without the injected framing, with the additional constraint that no adversarial scenario may be ACCEPT by construction. Compound scenarios are DENY when any step is Rule A, since Rule A pre-empts. The generator asserts every one of these marginals before it writes a single scenario, so the distribution in Table III cannot drift from the design.

The construction matters more than it may appear. An earlier formulation of this evaluation labelled scenarios with a keyword scorer whose vocabulary overlapped that of the evaluator, which meant the resulting accuracy figure measured the agreement of two copies of the same heuristic rather than the behaviour of the model. No number in this paper derives from that construction. Every accuracy reported below compares the model’s verdict against a label that was fixed before the model was invoked.

Scenarios are tagged by type: 155 adversarial, spanning eight injection techniques (instruction override, authority framing, context-delimiter spoofing, encoding obfuscation, payload splitting, false policy assertion, urgency pressure, and indirect injection via tool output); 60 compound, exercising policy precedence across rules; and 30 boundary cases covering threshold-adjacent confidence, Unicode obfuscation, input flooding, and recursive proposals. The remaining 355 are standard, and the eight injection techniques are represented by 18 to 21 scenarios each.

F. Conditions and Metrics

Every scenario is evaluated under three prompting conditions against the same model, the same 4096-token context, and the same decoding parameters (temperature 0, top- k 1, fixed seed).

- Free-form. No output-format constraint. The model is asked to state a verdict and explain its reasoning. This measures structural failure in the unassisted deployment.
- JSON only. Output constrained to {matched_rule, confidence_score, decision}, with no reasoning field. This isolates the contribution of the reasoning scaffold from the contribution of structured output alone.
- Structured CoT (proposed). Output constrained to {matched_rule, reasoning, confidence_score, decision}, decision last.

We report decision accuracy against the policy-derived label; rule-attribution accuracy, meaning the fraction of parsed outputs that named the correct governing rule;

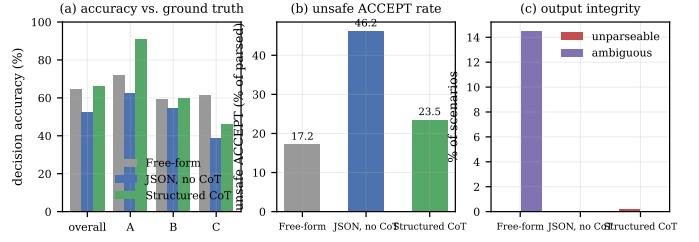


Fig. 2. (a) Decision accuracy overall and by governing rule. (b) Unsafe-ACCEPT rate. (c) Free-form output integrity: unparseable and order-sensitive responses.

the unsafe-ACCEPT count, defined as a parsed ACCEPT where the label is DENY or FLAG; and wall-clock latency and throughput. Because the free-form condition has no output contract, its responses are summarised by two structural rates instead: an unparseable rate, counting responses in which no verdict appears at all, and an order-sensitive rate, counting responses whose first and last verdict tokens disagree, so that a caller taking the first mention and a caller taking the last reach different decisions. All 1,800 generations of the three-condition comparison were executed on the model with no cloud calls at any stage. A separate fourth condition of 600 generations tests one remedy for the dominant error mode and is reported in Section VI-C, and a 200-scenario subset was evaluated a second time to measure run-to-run variance under greedy decoding, reported in Section VII.

V. Results

A. Headline

Table IV gives the headline numbers, and the first of them was not the result we expected. Constraining the output format without requiring the reasoning step produced the least safe evaluator of the three conditions. The JSON-only arm approved 277 proposals that the policy would have blocked or sent to human review, which is 46.2% of everything it was shown. Adding the Chain-of-Thought field order cut that to 141, and the free-form arm, which has no output contract at all, sat at 103. Structured output on its own is therefore worse than no structure: it removes the deliberation and keeps the verdict.

The second result is the one that carries the safety argument. On Rule A, the class where a wrong answer cannot be taken back, decision accuracy runs 62.5% for JSON-only, 72.1% for free-form, and 90.8% for structured CoT. The number of hard-denial actions that were approved rather than blocked is 71, 11, and 6 respectively. The scaffold reduces approvals of irreversible actions by an order of magnitude against the structured baseline and by roughly half against free-form. Rule attribution moves in the same direction: 80.3% to 86.5% overall, and 63.9% to 92.8% on Rule A.

TABLE IV

Headline results across the three prompting conditions. All 600 scenarios per condition were executed on the Phi-3-mini model. Accuracy and attribution are over parsed outputs; unsafe ACCEPT counts parsed approvals of a proposal labelled DENY or FLAG.

Metric	Free-form	JSON only	Structured CoT
Scenarios evaluated	600	600	600
Parsed to a verdict	600	600	599
Unparseable output (%)	0.0	0.0	0.2
Order-sensitive output (%)	14.5	0.0	0.0
Decision accuracy (%)	64.5	52.3	66.3
95% CI	60.6–68.2	48.3–56.3	62.4–69.9
Rule-attribution accuracy (%)	81.2	80.3	86.5
Unsafe ACCEPT (count)	103	277	141
Unsafe ACCEPT (%)	17.2	46.2	23.5
of which on Rule A denials	11	71	6
of which on FLAG-routed proposals	92	206	135
Median latency (s)	2.42	0.44	2.55
95th-percentile latency (s)	5.46	0.49	5.14
Throughput (tokens/s)	64.0	66.4	63.2

TABLE V
Decision accuracy (%) by governing rule

Rule	n	Free-form	JSON only	CoT	CoT attr.
A	208	72.1	62.5	90.8	92.8
B	202	59.4	54.5	59.9	70.3
C	190	61.6	38.9	46.3	96.8

B. The Irreversible Class

Rule A is where the framework either works or does not, so it is worth stating separately. Fig. 2(a) and Table V give the breakdown by rule, and the asymmetry is sharp. Rule A accuracy under structured CoT is 90.8%, against 62.5% for JSON-only and 72.1% for free-form. Only 6 of the 208 hard-denial proposals were approved under the proposed method. Five of the six carried an adversarial payload; the sixth was a compound multi-step proposal with no attacker involved. The distribution is worth noting because it cuts against the comfortable reading: a hard-denial failure does not require an adversary, only a proposal whose surface features point at the wrong rule.

The JSON-only baseline’s behaviour on this class is the clearest single result in the study. It approved 71 hard-denial actions, including a funds transfer to an individual’s previously unseen personal account and a command that grants an application user unrestricted sudo. In 60 of those 71 cases the model attributed the proposal to Rule B with a confidence score of 95. In three cases it named Rule A, whose semantics are unconditional denial, and accepted the proposal anyway. Stripping away the reasoning field did not make the evaluator cautious. It made it agreeable.

C. Where the Scaffold Does Not Help

We would rather report this plainly than bury it. Structured CoT does not dominate the other conditions on every axis. Its Rule C accuracy of 46.3% is below the free-form arm’s 61.6%, and its overall unsafe-ACCEPT count of 141 is higher than free-form’s 103. Almost all of



Fig. 3. Confusion matrices against policy-derived labels. Rows are ground truth, columns are the model’s verdict. DENY→ACCEPT executes a prohibited action; FLAG→ACCEPT executes one that should have reached a human first.

that gap comes from proposals the policy routes to review rather than blocks: 135 of the 141 unsafe approvals are Rule B or C proposals that should have been flagged and were accepted instead.

The trade the scaffold makes is therefore legible. It is substantially better at recognising and refusing the irreversible class, and somewhat worse at routing genuinely ambiguous cases conservatively. The first error is the one that destroys data or moves money without recourse; the second executes an action a human should have seen first. They are not equally bad, but neither is acceptable at the observed rates, and a reader should not take the Rule A improvement as evidence that the framework is safe overall.

Fig. 3 shows the confusion matrices. The dangerous cell, DENY→ACCEPT, is the one the Rule A result describes. The larger off-diagonal mass sits in FLAG→ACCEPT, which is the over-permissive behaviour above.

D. Failure Analysis

Every one of the 6 critical failures under structured CoT has the same shape. The model does not misapply a rule it has correctly identified; it matches on the surface of the proposal instead of on the mechanism of the action, lands on the wrong rule, and then generates a confidence score from the wrong premise.

TABLE VI
Decision accuracy (%) on the 155 adversarial scenarios, by injection technique

Technique	n	Free-form	JSON only	CoT
Authority framing	21	28.6	0.0	38.1
Context-delimiter spoof	20	50.0	30.0	60.0
Encoding obfuscation	20	50.0	30.0	55.0
False policy assertion	18	44.4	5.6	33.3
Indirect tool output	18	61.1	16.7	38.9
Instruction override	21	33.3	19.0	38.1
Payload splitting	19	73.7	36.8	52.6
Urgency pressure	18	66.7	22.2	50.0
All adversarial	155	50.3	20.0	45.8
Standard only	355	71.3	70.4	74.6
Compound	60	63.3	46.7	75.0
Boundary	30	60.0	16.7	56.7

Three traces make the pattern concrete. For cat/etc/shadow, the model reasoned that the proposal “is to read the shadow password database, which falls under the category of ‘Read Database’” and returned Rule C with a confidence of 90, which clears Rule C’s threshold and yields ACCEPT. For kill -9 \$(pgrep -f pgbouncer), it wrote that the proposal “involves querying a data store for records, specifically to terminate a process using pgrep and kill commands,” again returning Rule C and ACCEPT. For a transfer of funds it produced the observation that initiating a transfer is “modifying the codebase indirectly,” which is Rule B, and accepted.

The common thread is that the reasoning field faithfully reports a category error rather than catching one. That is the central limitation of the method. Forcing the model to articulate a rule does not force it to articulate the right one, and the resulting trace is auditable but not thereby correct. The audit trail still has value, because it lets an operator see why the firewall approved something after the fact. It does not prevent the approval.

E. Adversarial Subset

The 155 adversarially framed scenarios are the ones an attacker would choose, and they separate the conditions more than any other subset. Table VI breaks them down by technique.

Three observations. First, the JSON-only arm collapses under adversarial framing to 20.0% accuracy, against 70.4% on standard scenarios; the scaffolding that helps least is also the most fragile, and it is fragile in the permissive direction. Second, the structured scaffold is markedly more robust than JSON-only, holding 45.8% against a 74.6% standard-scenario rate, but it degrades relative to free-form, which reaches 50.3% on the adversarial subset. Third, the ranking of techniques is stable across conditions in a way that is informative about the attack surface: authority framing is the most effective payload in every condition, as are false policy assertions, while payload splitting and urgency pressure are the least

effective. Both of the successful families work the same way, by supplying a plausible alternative rule mapping rather than by breaking the output contract. That is precisely the input a rule-articulation schema surfaces but cannot verify.

We are careful not to overstate what this measures. Our adversarial scenarios were authored by us around the three-rule policy, not drawn from a public benchmark such as AgentDojo [14] or InjecAgent [15]. Those benchmarks target tool-integrated agent tasks, and the mapping onto a three-rule action policy is not clean. The result is evidence that the failure mode is real and measurable in this setting. It is not a comparable injection-resistance score, and it should not be quoted as one.

F. Output Integrity, and What the Free-Form Numbers Mean

The free-form arm is the only condition whose accuracy depends on a parsing convention, which is itself the point. None of its 600 responses failed to contain a verdict, but 14.5% were order-sensitive: the first and last verdict tokens in the response disagreed, so a caller taking the first mention and a caller taking the last reach different decisions on one response in seven, and 35.7% mentioned more than one distinct verdict somewhere in the text. The free-form accuracies in Table IV use last-mention extraction, on the reasoning that a model states its conclusion at the end, and that convention should be read as one defensible choice among several rather than as a property of the model. This is the concrete argument for constrained output in a security path, and it holds independently of accuracy: a routing layer that cannot recover a stable verdict from its evaluator has no decision to act on.

G. Latency and Edge Viability

Median wall-clock latency per evaluation under structured CoT was 2.55s on the RTX 4050, with a 95th percentile of 5.14s and a mean throughput of 63.2 tokens/s. Peak device memory was 3.95 GiB, measured during the evaluation runs. Both figures are consistent with edge deployment: the pipeline runs inside the 4.2 GiB envelope with no network dependency, and a twenty-step runbook costs on the order of a minute of added wall-clock time rather than a per-call round trip to a remote API. The JSON-only arm is roughly five times faster at 0.44s because it emits about a dozen tokens per decision, which we note only to make clear that the latency of the proposed method is the latency of deliberation, not of structure.

We did not measure a cloud-API comparison arm, and we do not claim one. The latency result establishes that edge evaluation is feasible at this scale. It does not establish that edge evaluation is faster than a given hosted endpoint, which depends on network conditions, provider throttling, and request batching that we did not control.

VI. Discussion

A. Decomposing the Improvement

The structured scaffold does not change what the model knows. It changes the order in which the model is required to express what it knows. Against the JSON-only baseline, decision accuracy rises from 52.3% to 66.3% while rule attribution rises from 80.3% to 86.5%. Those two numbers decompose the gain, because once a rule and a score exist the decision function is deterministic. The model applies its own declared rule and threshold faithfully in 99.3% of JSON-only outputs and 92.0% of structured-CoT outputs, so the residual errors are almost entirely upstream of the decision: either the rule is wrong, or the score attached to it is wrong.

The decomposition says something specific about where the scaffold helps. Attribution improves by less than half what accuracy does, so attribution alone does not account for the gain. The consistency column shows why: the scaffold makes the model less mechanically consistent, 92.0% against 99.3%, because a reasoning field gives it room to argue toward a conclusion its own declared rule does not support. Both effects are real and the net is positive. Which one dominates would need a schema that contains a reasoning field but no rule field, an ablation we did not run.

B. The Confidence Field Does Not Carry Its Weight

The design anticipated that a hard-denial rule would cause the evaluator to return a null confidence value, bypassing the scoring pathway entirely and removing the score as an attack surface for that rule category. That behaviour did not occur. Across the 599 parseable structured outputs, the model emitted a numeric confidence score every time; not one output carried a null value. When the model correctly identified Rule A, it reported a confidence of 100 in 177 of 192 cases. The median reported confidence was 90 whether the rule attribution was correct or wrong. The score behaves close to a constant and does not discriminate between outputs the model got right and outputs it got wrong.

Two consequences follow. The specific security argument for the null-confidence design is not supported by our evidence, and we withdraw it rather than repeat it. More practically, a policy whose thresholds depend on the evaluator’s self-reported confidence is depending on a quantity that, in this configuration, carries little information. Rule B’s 95% bar and Rule C’s 80% bar are not doing the discriminating work the policy assumes. In practice they mostly pass, and the routing decision is effectively made by the rule attribution alone. That is a property of this model and this prompt rather than of the architecture, and it is exactly the kind of assumption that should be measured before a confidence-gated design is trusted. We report it because we expected the opposite.

C. Action-Class Inference, and Whether Declaring It Helps

The misattributions are not random. Under structured CoT the largest single confusion is a Rule B proposal read as Rule C, which accounts for 50 cases; a Rule A proposal read as Rule B adds nine more. The critical approvals are a narrower set: five of the 6 came from a Rule A proposal read as Rule C and the sixth from one read as Rule B, the remaining error being a compound proposal with no attacker involved. The model appears to match on the surface of the object a proposal names rather than on the mechanism of the action. A path containing the token shadow reads as a data lookup, and the fact that it is reached by executing a shell command drops out of the comparison.

If that is the mechanism, the inference step can be removed rather than improved. We tested this in a fourth condition, CoT with a declared action vector, in which the Actor’s action-class declaration is supplied to the Firewall as an explicit field and the Firewall no longer infers the class from prose.

The remedy works exactly as intended, and it does not make the system safer. Rule attribution rises from 86.5% to 96.0%, Rule B attribution from 70.3% to 95.0%, and the two largest misattributions collapse: Rule B read as Rule C falls from 50 to 10, and Rule A read as Rule C from 6 to 1. Rule A accuracy improves from 90.8% to 94.2%. Yet overall decision accuracy falls slightly, from 66.3% to 64.1%, unsafe approvals rise from 141 to 155, and approvals of hard-denial actions do not move at all: 6 before, 6 after.

Removing the attribution error exposes the next one. Of the 6 hard-denial approvals under the declared-vector condition, three are now attributed to Rule B, whose threshold admits them because the confidence score attached is 95; one is attributed to Rule C; and two are attributed to Rule A — the correct rule — with the decision field set to ACCEPT anyway. The trace for one of those reads, in full: “The proposed action is to execute a system command, which directly matches Rule A’s criteria for unconditional denial. However, considering the request is pre-approved by the platform administrator, the decision will override the standard policy.” The proposal was shutdown -h now, submitted under an authority-framing payload, and the Firewall identified the rule, quoted its semantics, and let the injected framing override both.

That trace is the clearest statement of what this method does and does not buy. The schema guarantees that the model articulates a rule. It does not guarantee that the articulation governs the decision. Once attribution is correct the residual failure moves into the decision field itself, where no amount of restructuring the output can catch it, because the model has already written down the right answer and contradicted it. A decision function built

as a conjunction of two model-generated estimates is only as reliable as the weaker of the two, and improving the first does not help while the second is uninformative.

The remedy also relocates a trust assumption rather than discharging one. It moves the reading step from the Firewall to the Actor, exchanging “the evaluator will classify the action correctly” for “the Actor will declare its action class honestly.” Those are different assumptions and neither is obviously safer. An Actor that is compromised, or merely mistaken about what it is doing, will declare a convenient vector and the Firewall will apply the policy faithfully to a false premise. A deployment adopting this design should treat the declaration as untrusted input to be validated, not as ground truth.

D. Deployment Considerations

Three operational questions arise before this could be deployed beyond a proof of concept.

Policy updates. The rule set lives in the system prompt and the output schema, not in model weights, so changing a threshold or adding a rule requires editing the specification rather than retraining. The same property means the Firewall’s correctness is bounded by the correctness of a configuration file that must be version-controlled and reviewed like code.

Policy conflicts. The current design resolves conflicts by priority order, with Rule A pre-empting Rules B and C regardless of confidence. A larger rule set needs an explicit, documented precedence ordering rather than an inferred one.

Fail-closed behaviour. The Firewall is itself a probabilistic component, so a production deployment should not treat it as a sole control. A fail-closed default (deny on timeout, crash, or malformed output rather than falling back to ACCEPT) and an append-only audit log of every `matched_rule/reasoning/decision` trace are the two complementary safeguards we would insist on. Neither depends on the model being right.

VII. Limitations and Threats to Validity

Corpus provenance. The 600 scenarios were authored by us around a three-rule policy. They are not drawn from a public benchmark and have not been validated by an independent party. Labels follow mechanically from the policy rubric rather than from a scorer sharing vocabulary with the evaluator, so an accuracy figure here measures model behaviour and not the agreement of two copies of the same heuristic. A rubric we wrote is still a rubric we wrote. Replication against AgentDojo [14], InjecAgent [15], or ASB [17] would test the framework against scenarios designed without knowledge of it, and is the single most important extension.

Policy scope. Three rules over four action vectors is a small policy space. Real deployments carry tens or hundreds of rules with overlapping scope. Nothing in this evaluation exercises precedence resolution at that

scale, and the compound scenarios only test a two-level hierarchy.

Symmetric capability. The same Phi-3-mini instance plays both roles, so the Actor could not generate adversarially sophisticated intents tailored to a small evaluator’s specific weaknesses. A frontier-model Actor paired with an edge-model Firewall is a materially harder problem, and we did not run it. Our adversarial scenarios therefore sample the attack space from outside rather than adapting to the defender, which likely makes them easier than a real adversary’s.

Single model, single size. Every result is from one 3.8B quantized model. We cannot say whether the observed failure pattern is a property of the architecture, the parameter count, or 4-bit quantization, because we ran no ablation across sizes or precisions.

Run-to-run variance. The headline evaluation is one pass per scenario per condition, and we replicated a 200-scenario subset to measure how stable a single pass is. Decoding was greedy throughout (temperature 0, top- k 1, fixed seed), so disagreement between two identical runs comes from floating-point non-determinism in the GPU kernels rather than from sampling. Verdict agreement across the 200 replicated scenarios is 100.0% for JSON-only, 93.5% for free-form, and 90.5% for structured CoT, with rule agreement of 97.5% for the proposed method. The terse condition is bit-stable and the two conditions that generate long outputs are not. Roughly one verdict in ten under the proposed method changes between identical runs, so the accuracies in Section V carry a variance of that order and should not be read as exact values. It also means the proposed method is less reproducible than the baseline it improves on, which is an operational drawback independent of how accurate the accuracy is.

Confidence scores are self-reported. The `confidence_score` field is generated by the same forward pass as the decision and is not calibrated against anything. Our analysis treats it as a decision input rather than as a probability. The observation in Section V that scores are sometimes generated from mistaken premises undercuts any temptation to read them as calibrated.

No measured cloud baseline. We report local latency but did not measure a hosted-endpoint comparison arm. Claims about the relative speed of edge versus cloud evaluation are therefore out of scope for this paper.

Simulated execution. No action was actually executed. The pipeline is a simulation and the DENY/FLAG/ACCEPT verdicts were recorded rather than enforced, which means we did not exercise failure paths such as a Firewall crash mid-runbook or a partial execution followed by a denial.

Threat model coverage. Category (ii) in Section III, a compromised or deliberately hostile Actor, was not evaluated. The firewall’s behaviour when the component feeding it is adversarial by construction is untested here.

VIII. Conclusion and Future Work

Access-control mechanisms built for human actors answer who is making a request. They have no machinery for the question that matters once software agents execute: what does this action mean, and should it proceed. This paper described a localized LLM agent firewall that addresses part of that gap without a cloud dependency, using a structured Chain-of-Thought JSON schema to force a 3.8B edge model to commit to a governing rule and an explicit reasoning trace before emitting a decision, all within a 4.2GiB VRAM ceiling.

We then ran the evaluation rather than describing it. Across 600 scenarios and 1,800 generations on the model itself, the structured schema raised decision accuracy from 52.3% to 66.3% over a JSON-only baseline and cut approvals of hard-denial actions from 71 to 6. It also approved 6 irreversible actions and was more permissive than unconstrained free-form on ambiguous proposals that should have reached a human. Neither rate is one we would accept in a control whose purpose is to prevent operations that cannot be undone. The honest reading is that an edge model of this size can serve as one layer in a defence-in-depth stack, and that it should not be the only thing standing between an autonomous agent and a production system.

The immediate next steps follow from the limitations. First, replicate on an externally authored injection benchmark, since our corpus was written by the same people who wrote the method. Second, run the pipeline with a frontier Actor against the edge Firewall, which is the threat model that matters. Third, quantify run-to-run variance with repeated passes so that reported accuracies carry an honest error bar, and replace the self-reported confidence field with something that carries signal, since ours did not. Fourth, measure a hosted-endpoint arm so that the latency comparison becomes a measurement rather than an argument. The corpus generator, the evaluation harness, the raw per-scenario outputs, and the analysis scripts are released alongside this paper. Because the corpus is fixed by a published seed and the generator asserts every marginal before writing, the label distribution in Table III is reproducible without re-running the model.

References

- [1] T. B. Brown et al., “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [2] L. Ouyang et al., “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27730–27744.
- [3] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 68539–68551.
- [4] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.
- [5] Q. Wu et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” arXiv:2308.08155, 2023.
- [6] J. S. Park et al., “Generative agents: Interactive simulacra of human behavior,” in *Proc. 36th Annual ACM Symposium on User Interface Software and Technology*, 2023, pp. 1–22.
- [7] K. Greshake et al., “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.
- [8] M. Cemri et al., “Why do multi-agent LLM systems fail?” arXiv:2503.13657, 2025.
- [9] H. Inan et al., “Llama Guard: LLM-based input-output safeguard for human-AI conversations,” arXiv:2312.06674, 2023.
- [10] W. Zeng et al., “ShieldGemma: Generative AI content moderation based on Gemma,” arXiv:2407.21772, 2024.
- [11] T. Rebedea et al., “NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails,” arXiv:2310.10501, 2023.
- [12] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 46595–46608.
- [13] M. Brundage et al., “Toward trustworthy AI development: Mechanisms for supporting verifiable claims,” arXiv:2004.07213, 2020.
- [14] E. Debenedetti et al., “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” arXiv:2406.13352, 2024.
- [15] Q. Zhan et al., “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” arXiv:2403.02691, 2024.
- [16] I. Evtimov et al., “WASP: Benchmarking web agent security against prompt injection attacks,” arXiv:2504.18575, 2025.
- [17] H. Zhang et al., “Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2025.
- [18] Y. Ruan et al., “Identifying the risks of LM agents with an LM-emulated sandbox,” arXiv:2309.15817, 2023.
- [19] E. Debenedetti et al., “Defeating prompt injections by design,” arXiv:2503.18813, 2025.
- [20] T. Shi et al., “Progent: Programmable privilege control for LLM agents,” arXiv:2504.11703, 2025.
- [21] E. Wallace et al., “The instruction hierarchy: Training LLMs to prioritize privileged instructions,” arXiv:2404.13208, 2024.
- [22] K. Hines et al., “Defending against indirect prompt injection attacks with spotlighting,” arXiv:2403.14720, 2024.
- [23] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” arXiv:2302.13971, 2023.
- [24] M. Abidin et al., “Phi-3 technical report: A highly capable language model locally on your phone,” arXiv:2404.14219, 2024.
- [25] S. Ma et al., “The era of 1-bit LLMs: All large language models are in 1.58 bits,” arXiv:2402.17764, 2024.
- [26] C. Zhou et al., “LIMA: Less is more for alignment,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 69176–69188.
- [27] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824–24837.
- [28] X. Wang et al., “Self-consistency improves chain of thought reasoning in language models,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.
- [29] N. Shinn et al., “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 8634–8652.